



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático 4 - Sistemas Distribuídos

HENRIQUE ALVES CAMPOS [EF04231]

HENRIQUE DE SOUZA CAMPOS [EF04673]

HERON FILLIPE SILVEIRA SANTOS [EF04211]

Sumário

Sumário.....	2
1. Introdução.....	3
2. Arquitetura do Sistema.....	3
2.1. Servidor.....	3
2.2. Cliente.....	4
3. Modelo de Objetos Distribuídos.....	5
3.1. Registro e Descoberta via Name Server.....	6
3.2. Interface Remota dos Objetos Distribuídos.....	7
4. Concorrência e Sincronização.....	9
4.1. Threads no Servidor.....	9
4.2. Proteção do Estado Compartilhado.....	9
4.3. Polling e Atualização da Interface.....	10
5. Execução do Sistema.....	10
6. Conclusão.....	12
Referências.....	13

1. Introdução

Esta quarta e última etapa do trabalho prático conclui o desenvolvimento do sistema distribuído GridCity, iniciado nas partes anteriores. Enquanto a Parte 3 implementou o sistema utilizando diretamente a API de Sockets, com comunicação baseada em troca de mensagens JSON e gerenciamento manual do protocolo, esta fase reimplementa o mesmo sistema utilizando o middleware Pyro5, que adota a visão de objetos distribuídos.

O objetivo desta etapa é evidenciar as diferenças práticas entre as duas abordagens. Com o Pyro5, as entidades do sistema são representadas como objetos remotos cujos métodos podem ser invocados diretamente pelo cliente, sem que o programador precise definir estruturas de mensagens, tratar fragmentação de pacotes ou gerenciar conexões individuais. Essa abstração desloca o foco do desenvolvimento para a lógica de negócio do sistema.

A implementação foi realizada integralmente em Python, utilizando Pyro5 para comunicação entre objetos distribuídos, PyQt6 para a interface gráfica, SQLite para persistência do estado e threads para gerenciamento de concorrência. O código-fonte do projeto está disponível no repositório público: <https://github.com/alveshenrique/CCF-355-SistemasDistribuidos>

2. Arquitetura do Sistema

O GridCity mantém a arquitetura cliente-servidor centralizada definida nas etapas anteriores. Um servidor único expõe o estado global da grade como dois objetos remotos acessíveis via Pyro5, e múltiplos clientes interagem com esses objetos invocando métodos remotos diretamente, sem necessidade de um protocolo de mensagens explícito.

2.1 Servidor

O servidor é composto por três módulos principais e um modelo de dados. O ponto de entrada é responsável por inicializar o daemon Pyro5 e registrar ambos os objetos remotos no Name Server. Os outros dois módulos representam os objetos remotos propriamente ditos: um encapsula toda a lógica relacionada à grade e à persistência, e o outro gerencia exclusivamente o controle de presença dos usuários conectados. Essa separação em dois objetos com responsabilidades distintas reflete o princípio de

granularidade de objetos no paradigma RMI, evitando a concentração de responsabilidades em um único objeto remoto decisão que também foi destacada como ponto de melhoria durante o desenvolvimento.

server.py

Ponto de entrada do servidor. Responsável por instanciar o GridManager e o SessionManager, registrar ambos os objetos no daemon do Pyro5 com os identificadores `gridcity.grid` e `gridcity.session` e publicar suas URIs no Name Server. Após o registro, inicia o loop de requisições do daemon, que passa a aguardar e despachar chamadas remotas dos clientes.

grid_manager.py

Responsável pelo gerenciamento do estado global da grade 7×7 e pela persistência dos dados em banco SQLite. Ao ser instanciado, cria a tabela de estruturas caso não exista e carrega o estado persistido para a memória. Expõe três métodos remotos:

- `place_structure(x, y, structure_type, author)` — valida se a posição está livre, insere a estrutura na grade em memória e persiste no banco
- `remove_structure(x, y)` — valida se a posição está ocupada, remove da grade em memória e do banco
- `get_state()` — retorna a grade serializada como lista de listas de dicionários

session_manager.py

Responsável pelo controle dos usuários conectados ao sistema. Mantém um dicionário interno de sessões ativas e expõe três métodos remotos:

- `join(username)` — registra o usuário na lista de conectados
- `leave(username)` — remove o usuário da lista de conectados
- `get_users()` — retorna um dicionário com o número de usuários conectados e a lista de identificadores

models/structure.py

Define a classe `Structure`, que representa uma estrutura alocada em uma célula da grade. Armazena `structure_type`, `author` e `timestamp`, e fornece o método `to_dict()` para serialização durante as respostas remotas.

2.2 Cliente

O cliente é composto por três módulos com responsabilidades bem separadas. Em relação à Parte 3, os módulos `client_handler.py` e `protocol.py` foram removidos, pois o Pyro5 elimina a necessidade de um handler por conexão e de funções dedicadas à construção de mensagens.

client.py

Ponto de entrada do cliente. Sua função é exclusivamente inicializar a aplicação e invocar a função `main()` definida em `ui.py`.

network.py

Responsável pela comunicação com o servidor via Pyro5. Composto por duas classes:

- `NetworkSignals` — define os sinais Qt utilizados para comunicação segura entre a thread de rede e a interface gráfica: `state_updated`, `users_updated` e `response_received`
- `NetworkClient` — gerencia a conexão com os objetos remotos por meio de dois proxies Pyro5, um apontando para `gridcity.grid` e outro para `gridcity.session`. Ao conectar, invoca `join()` no servidor e inicia uma thread daemon responsável pelo polling periódico do estado e da lista de usuários. As operações `place_structure()` e `remove_structure()` são invocadas diretamente no proxy, e os resultados são emitidos como sinais Qt para atualização da interface

ui.py

Responsável pela interface gráfica, construída integralmente com PyQt6. Mantém os mesmos três componentes da Parte 3: `LoginDialog`, `CityGrid` e `MainWindow`. Não houve alterações relevantes neste módulo em relação à implementação anterior.

3. Modelo de Objetos Distribuídos

No Pyro5, a comunicação entre processos é abstraída pelo middleware, que permite ao cliente invocar métodos em objetos remotos como se fossem objetos locais.

Diferentemente da Parte 3, onde toda a comunicação precisava ser construída manualmente por meio de mensagens JSON, aqui o programador define uma classe Python comum e a decora com `@Pyro5.api.expose`. A partir dessa decoração, o middleware assume a responsabilidade de serializar os parâmetros, transmiti-los pela rede, desserializá-los no servidor, executar o método e retornar o resultado ao chamador. Todo esse processo ocorre de forma transparente, sem que o código da aplicação precise conhecer detalhes do transporte subjacente. No GridCity, essa abordagem foi aplicada a dois objetos remotos com responsabilidades distintas, o `GridManager` e o `SessionManager`, cada um registrado com um identificador próprio no Name Server.

3.1 Registro e Descoberta via Name Server

O Name Server é um componente central do Pyro5 que funciona como um serviço de diretório: o servidor registra seus objetos remotos com nomes legíveis, e os clientes utilizam esses nomes para localizar e conectar aos objetos sem precisar conhecer previamente o endereço IP ou a porta em uso.

A sequência de inicialização do sistema segue obrigatoriamente a seguinte ordem:

1. O Name Server do Pyro5 é iniciado e passa a aguardar registros
2. O servidor instancia o `GridManager` e o `SessionManager`, registra ambos os objetos no daemon Pyro5 com os identificadores `gridcity.grid` e `gridcity.session` e publica suas URIs no Name Server
3. O cliente cria dois proxies, um apontando para `PYRONAME:gridcity.grid` e outro para `PYRONAME:gridcity.session`, e o Pyro5 consulta o Name Server para resolver cada nome e estabelecer as conexões com os objetos remotos correspondentes

Essa ordem é obrigatória. Se o servidor tentar registrar os objetos antes do Name Server estar ativo, ou se o cliente tentar resolver os nomes antes do servidor estar registrado, a inicialização falhará.

O diagrama abaixo ilustra essa sequência de inicialização e o registro dos dois objetos remotos no Name Server.

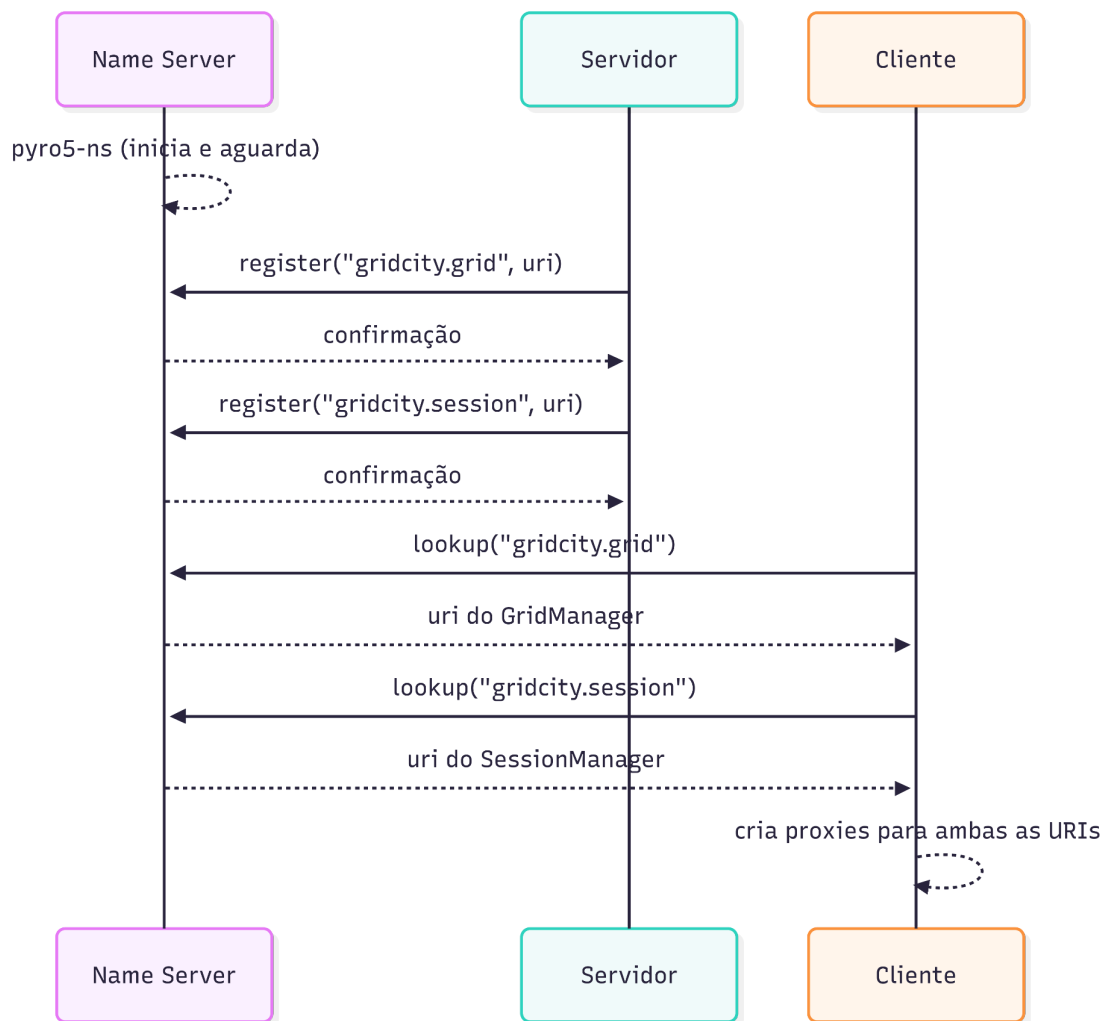


Figura 1 - Inicialização e Registro dos Objetos Remotos no Name Server

3.2 Interface Remota dos Objetos Distribuídos

O sistema expõe dois objetos remotos independentes, cada um com responsabilidade bem definida. O cliente interage com ambos exclusivamente por meio de proxies Pyro5, que interceptam cada chamada de método, serializam os argumentos, transmitem ao servidor, aguardam o retorno e o entregam ao chamador de forma transparente.

GridManager

Responsável por toda a lógica relacionada à grade 7×7 . É o objeto remoto central do sistema, acessado pelo cliente via `PYRONAME:gridcity.grid`. Expõe os seguintes métodos:

- `place_structure(x, y, structure_type, author)` — valida se a posição está livre, aloca a estrutura na grade em memória, persiste no banco e

retorna uma tupla indicando sucesso ou falha com mensagem descritiva

- `remove_structure(x, y)` — valida se a posição está ocupada, remove a estrutura da grade em memória e do banco, e retorna uma tupla indicando sucesso ou falha
- `get_state()` — retorna o estado completo da grade 7×7 como lista de listas de dicionários, com cada célula sendo `null` se vazia ou um objeto com `type`, `author` e `time` se ocupada

SessionManager

Responsável pelo controle de presença dos usuários conectados. É acessado pelo cliente via `PYRONAME:gridcity.session`. Expõe os seguintes métodos:

- `join(username)` — registra o usuário na lista de conectados no momento da conexão
- `leave(username)` — remove o usuário da lista de conectados no momento da desconexão
- `get_users()` — retorna um dicionário com o número de usuários conectados e a lista de identificadores

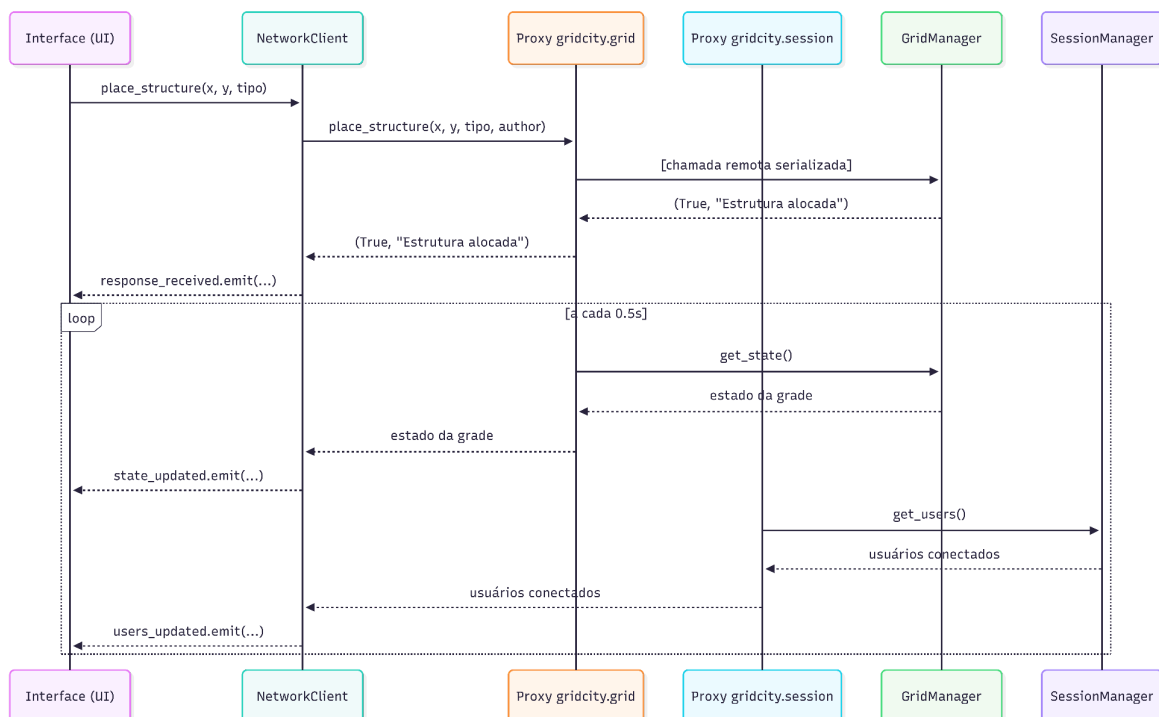


Figura 2 - Fluxo de Operação e Polling com Dois Proxies

4. Concorrência e Sincronização

O uso de threads na Parte 4 é mais simples do que na Parte 3. O Pyro5 gerencia internamente o atendimento de múltiplos clientes, eliminando a necessidade de criar e controlar threads de conexão manualmente. O programador precisa lidar com threads apenas para proteger o estado compartilhado no servidor e para manter a interface gráfica responsiva no cliente.

4.1 Threads no Servidor

O daemon do Pyro5 gerencia internamente o recebimento e despacho de chamadas remotas, abstraindo o controle de conexões que na Parte 3 exigia uma thread dedicada por cliente. O servidor não cria threads explicitamente para atender clientes. Ainda assim, como múltiplas chamadas remotas podem ser despachadas concorrentemente pelo daemon, o acesso ao estado compartilhado precisa ser protegido.

4.2 Proteção do Estado Compartilhado

O principal desafio de concorrência no servidor continua sendo o acesso simultâneo ao estado compartilhado por múltiplas chamadas remotas despachadas concorrentemente pelo daemon do Pyro5. Com a separação em dois objetos remotos independentes, cada um possui seu próprio `threading.Lock`, o que representa uma melhoria em relação à versão anterior: operações na grade e operações de sessão não competem pelo mesmo lock, reduzindo a contenção e tornando o sistema mais eficiente sob carga concorrente.

O `GridManager` utiliza seu lock para proteger todas as leituras e escritas na grade em memória, garantindo que apenas uma chamada por vez execute a verificação e a modificação de uma célula. O `SessionManager` utiliza seu lock de forma independente para proteger o dicionário de usuários conectados, garantindo que operações de entrada e saída de usuários sejam atômicas.

Em ambos os casos, a escrita no banco SQLite é realizada fora do lock intencionalmente, pois o SQLite possui seu próprio mecanismo de controle de concorrência, e liberar o lock antes da escrita no banco reduz o tempo de contenção entre as chamadas.

4.3 Polling e Atualização da Interface

No lado do cliente, o desafio é manter a interface atualizada com o estado do servidor sem bloquear a thread principal do PyQt6. A solução adotada é diferente da Parte 3: em vez de receber atualizações passivamente via broadcast, o cliente consulta ativamente o servidor a cada 0,5 segundos por meio de uma thread daemon dedicada, chamando `get_state()` e `get_users()` no proxy Pyro5 e emitindo os sinais Qt correspondentes com os dados recebidos.

Assim como na Parte 3, o mecanismo de sinais e slots do Qt garante que as atualizações da interface sejam executadas na thread principal da UI, de forma segura e sem necessidade de locks explícitos no lado do cliente.

5. Execução do Sistema

A execução do GridCity com Pyro5 exige que três processos sejam iniciados em ordem específica. Ao contrário da Parte 3, onde apenas o servidor e o cliente precisavam ser iniciados, aqui o Name Server do Pyro5 deve estar ativo antes de qualquer outro processo.

Requisitos

- Ubuntu 22.04
- Python 3.10 ou superior
- Dependências do Sistema:

Shell

```
sudo apt-get install libxcb-cursor0
```

Passo 1 - Clonar o repositório

Shell

```
git clone https://github.com/alveshenrique/CCF-355-SistemasDistribuidos.git  
cd CCF-355-SistemasDistribuidos
```

Passo 2 - Acessar a pasta da parte 4

Shell

```
cd parte4
```

Passo 3 - Instalar dependências

```
Shell  
make install
```

Passo 4 - Iniciar o Name Server do Pyro5

Em um terminal dedicado:

```
Shell  
make nameserver
```

O Name Server deve permanecer ativo durante toda a execução do sistema. Encerrá-lo interrompe a comunicação entre cliente e servidor.

Passo 5 - Iniciar o servidor

Em outro terminal:

```
Shell  
make server
```

O servidor só deve ser iniciado após o Name Server estar ativo. Caso contrário, o registro do objeto remoto falhará.

Passo 6 - Iniciar o cliente

Em outro terminal:

```
Shell  
make client
```

Para simular múltiplos usuários, abra terminais adicionais e execute `make client` em cada um.

Resetar o mapa:

```
Shell  
rm cidade.db
```

6. Conclusão

A reimplementação do GridCity utilizando o middleware Pyro5 permitiu experienciar de forma direta as diferenças entre as duas abordagens de desenvolvimento de sistemas distribuídos trabalhadas neste projeto.

A principal vantagem percebida com o Pyro5 foi a eliminação de toda a camada de protocolo que precisou ser construída manualmente na Parte 3. A definição do formato das mensagens, o tratamento de fragmentação TCP, o buffer acumulativo e o gerenciamento de conexões individuais deixaram de existir como responsabilidade do programador. No lugar disso, o cliente simplesmente invoca métodos no objeto remoto como se fossem métodos locais, e o middleware cuida de toda a comunicação subjacente.

A adoção do Name Server introduziu uma dependência adicional na inicialização do sistema que não existia na Parte 3. É necessário garantir que o Name Server esteja ativo antes do servidor e do cliente, o que adiciona uma etapa ao processo de execução e um ponto de falha que precisa ser considerado.

Uma limitação identificada na implementação foi a ausência de um mecanismo de broadcast nativo no Pyro5. Na Parte 3, o servidor propagava atualizações ativamente para todos os clientes conectados. Com o Pyro5, essa funcionalidade precisou ser substituída por polling periódico no cliente, com consultas a cada 0,5 segundos. Essa abordagem é funcional para o escopo do trabalho, mas introduz uma latência mínima nas atualizações e aumenta o número de chamadas remotas ao servidor.

De modo geral, o paralelo entre as duas implementações evidencia com clareza o papel dos middlewares no desenvolvimento de sistemas distribuídos. A Parte 3 exigiu maior controle e atenção a detalhes de baixo nível, mas ofereceu visibilidade total sobre o que acontece na comunicação. A Parte 4 reduziu significativamente a complexidade do código, deslocando o foco para a lógica de negócio, mas introduziu dependências e abstrações que o programador precisa compreender para depurar e evoluir o sistema.

Referências

- [1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2. ed. São Paulo: Pearson, 2007.
- [2] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. Distributed Systems: Concepts and Design. 5. ed. Boston: Addison-Wesley, 2011.
- [3] PYTHON SOFTWARE FOUNDATION. threading — Thread-based parallelism. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 2026.
- [4] PYTHON SOFTWARE FOUNDATION. sqlite3 — DB-API 2.0 interface for SQLite databases. Disponível em: <https://docs.python.org/3/library/sqlite3.html>. Acesso em: 2026.
- [5] RIVERBANK COMPUTING. PyQt6 Reference Guide. Disponível em: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>. Acesso em: 2026.
- [6] IRMEN DE JONG. Pyro5 — Python Remote Objects. Disponível em: <https://pyro5.readthedocs.io>. Acesso em: 2026.